1 Editorial: Cartonaşe (Cards)

1.1 Problem Overview

This problem presents three different subproblems, identified by the value of c (1, 2, or 3). Given n card values $v_1, v_2, \ldots, v_n$, we need to solve different queries based on the value of c.

- **Input format:** The first line contains c and n. The second line contains n integers $v_1, v_2, \ldots, v_n$.
- The behavior depends entirely on which case ($c = 1$, 2, or 3) is specified.

1.2 Case 1: Counting Smaller Elements

Task: Given a position poz, count how many elements among $v_1, v_2, \ldots, v_{\text{poz}}$ are strictly less than v_{poz}.

Key Observation: We need to find the first index i in the prefix where $v_i \geq v_{\text{poz}}$. All indices before i (i.e., $1, 2, \ldots, i-1$) satisfy $v_j < v_{\text{poz}}$, which is exactly what we need to count.

Algorithm: The solution performs a linear scan from position 1 to poz, breaking when it finds $v_i \geq v_{\text{poz}}$. The answer is $i - 1$.

Complexity: $O(\text{poz})$ time, $O(1)$ additional memory.

1.3 Case 2: Permutation Prefixes

Task: Find all positions i such that the prefix $v_1, v_2, \ldots, v_i$ contains exactly the numbers $\{1, 2, \ldots, i\}$ (in any order).

Key Observation: Let maxim $= \max(v_1, v_2, \ldots, v_i)$ be the maximum value in the prefix. If the prefix forms a permutation of $\{1, 2, \ldots, i\}$, then we must have:

$$\text{maxim} = i \quad \text{and} \quad \text{all values} \in \{1, 2, \ldots, i\} \tag{1}$$

Since we're scanning left to right and tracking the running maximum, whenever maxim $== i$, the prefix must be a valid permutation (the prefix contains i distinct positive integers, with maximum exactly i).

Algorithm: Iterate through the array while maintaining maxim $= \max(v_1, \ldots, v_i)$. Whenever maxim $== i$, output i.

Complexity: $O(n)$ time, $O(1)$ additional memory.

```
maxim = 0;
nr = 0;
for (i = 1; i <= n; i++) {
    if (v[i] > maxim)
        maxim = v[i];
    if (maxim == i) {
        nr++;
        if (nr != 1)
```

```
            fout << "␣";
        fout << i;
    }
}
```

1.4 Case 3: Two Largest Elements Condition

Task: Find all positions i such that there exists a subset of $\{v_1, v_2, \ldots, v_i\}$ that equals $\{1, 2, \ldots, i\}$.

Key Observation: This is equivalent to checking whether the prefix can form a permutation of $\{1, 2, \ldots, i\}$ by possibly removing some elements.

Let maxim_1 be the largest value and maxim_2 be the second largest value in the prefix. For the prefix to contain a valid permutation:

$$\text{maxim}_1 > i \quad \text{(some element exceeds } i\text{)} \tag{2}$$

$$\text{maxim}_2 \leq i \quad \text{(all other elements are within range)} \tag{3}$$

The second condition ensures that after potentially removing the single large element maxim_1, the remaining elements can form exactly $\{1, 2, \ldots, i\}$ if they are all in the correct range.

Algorithm: Track the two largest values while scanning. Output position i when both conditions are satisfied.

Complexity: $O(n)$ time, $O(1)$ additional memory.

```
maxim1 = 0;
maxim2 = 0;
nr = 0;
for (i = 1; i <= n; i++) {
    if (v[i] > maxim1) {
        maxim2 = maxim1;
        maxim1 = v[i];
    } else if (v[i] > maxim2)
        maxim2 = v[i];
    if (maxim1 > i && maxim2 <= i) {
        nr++;
        if (nr != 1)
            fout << "␣";
        fout << i;
    }
}
```

1.5 Summary of Cases

Case	Goal	Key Insight
$c = 1$	Count elements $< v_{\text{poz}}$ in prefix	Linear scan to find first $\geq$
$c = 2$	Find permutation prefixes	$\max = i$ implies valid permutation
$c = 3$	Find positions with subset condition	Largest $> i$ and 2nd largest $\leq i$

1.6 Overall Complexity

The solution runs in $O(n)$ time for all cases (with $O(\text{poz})$ for case 1 in the worst case). All cases use $O(1)$ additional memory beyond the input array.

The array $f[\text{DIM}]$ declared in the code is used for frequency counting (though not directly utilized in the visible solution logic), demonstrating an alternative approach that could be used for case 1 by building a frequency histogram.